Fenwick Tree (Binary Indexed Tree)

Fenwick Tree is a data structure that stores partial aggregates in a hierarchical way so that prefix queries and point updates both take logarithmic time.

Fenwick Tree answers only one type of question efficiently:

"What is the aggregate from index 1 to index x?"

How Does It Store Information? (High-Level)

Instead of storing:

```
sum(1,1)
sum(1,2)
sum(1,3)
sum(1,4)
...
```

Fenwick Tree stores:

Aggregates of blocks whose sizes are powers of two

These blocks:

- End at specific positions
- Cover aligned chunks
- Can be combined to form any prefix

Think in Terms of Blocks, Not Indices

Imagine your array as a number line.

Fenwick Tree breaks this line into overlapping power-of-two blocks.

Each position remembers:

- The sum of a block ending there
- The block size is the largest power of two that fits

So instead of remembering everything, we remember strategic checkpoints.

Why Powers of Two?

Because powers of two allow:

- Clean splitting
- Efficient jumping
- Logarithmic depth
- Binary navigation

This is the same reason:

- Binary search works
- Segment trees work
- Binary lifting works

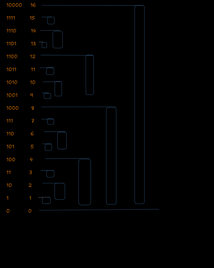
Fenwick Tree is a flat, iterative version of this idea.

Lowest Set Bit (LSB)

For a number x:

```
LSB(x) = x & (-x)
```

The LSB tells us how big a chunk this index is responsible for.



Querying: "Walking Backward in Blocks"

When you ask:

What is the total up to position x?

Fenwick Tree answers:

"I'll cover that range using the fewest blocks possible, starting from the end"

How

1. Take the biggest block ending at x
2. Jump back by its size
3. Repeat

Each step removes at least half of the remaining range
→ O(log n) steps

Example: sum(1, 7)

```
Start at 7
+ bit[7] covers [7..7]
+ 7 - LSB(7) = 6

+ bit[6] covers [5..6]
+ 6 - LSB(6) = 4

+ bit[4] covers [1..4]
+ 4 - LSB(4) = 0
```

Code

```
int query(int idx) {
    int sum = 0;
    while (idx > 0) {
        sum += bit[idx];
        idx -= idx & (-idx);
    }
    return sum;
}
```

Updating: "Who Cares About This Change?"

When a single value changes, Fenwick Tree asks:

"Which blocks include this position?"

Answer:

- All blocks that start before or at this index
- And end after or at this index

Those blocks live at:

```
index
index + next power of two
index + larger power of two
...
```

Again:

- Each jump doubles the block size
- Number of affected blocks = O(log n)

Example: update index 3

```
Idx = 3
+ bit[3] += delta
+ 3 + LSB(3) = 4

+ bit[4] += delta
+ 4 + LSB(4) = 8

+ bit[8] += delta
+ 8 + LSB(8) = 16 (stop)
```

Code

```
void update(int idx, int delta) {
    while (idx <= n) {
        bit[idx] += delta;
        idx += idx & (-idx);
    }
}
```

Range Sum Query

Once you have prefix sums:

```
sum(l, r) = query(r) - query(l - 1)
```

Complete Fenwick Tree Implementation (C++)

```
struct Fenwick {
    int n;
    vector<long long> bit;

    Fenwick(int n) {
        this->n = n;
        bit.assign(n + 1, 0);
    }

    void update(int idx, long long delta) {
        while (idx <= n) {
            bit[idx] += delta;
            idx += idx & (-idx);
        }
    }

    long long query(int idx) {
        long long sum = 0;
        while (idx > 0) {
            sum += bit[idx];
            idx -= idx & (-idx);
        }
        return sum;
    }

    long long rangeQuery(int l, int r) {
        return query(r) - query(l - 1);
    }
};
```

Building Fenwick Tree

Option 1: Update One-by-One (Simple)

```
for (int i = 1; i <= n; i++) {
    fenwick.update(i, A[i]);
}
```

⏱ O(n log n)

Option 2: Faster Build (Advanced)

```
for (int i = 1, j; i <= n; i++) {
    bit[i] += A[i];
    j = i + (i & -i);
    if (j <= n) {
        bit[j] += bit[i];
    }
}
```

⏱ O(n)